Tiny code (SQL) - Fine-Tuning Lightweight Language Models for BigQuery SQL Generation

Santa Das Karmakar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
sdaskarmakar@sandiego.edu
santa-dask/aai-590-tinysql

Sethuraman Krishnasamy
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
skrishnasamy@sandiego.edu
santa-dask/aai-590-tinysql

Sanjay Kumar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
sanjaykumar@sandiego.edu
sanjaykr33/aai-590-tinysql

Introduction

In the contemporary landscape of generative artificial intelligence, large-scale proprietary models such as GPT-5 and Gemini 3.0 Pro have established an extraordinarily high benchmark for code generation. However, for the modern enterprise, these "frontier" models present formidable barriers to entry that extend beyond mere subscription costs. These include prohibitive per-token inference expenses, high operational latency, and most critically the existential security risk associated with transmitting sensitive internal database schemas to external third-party cloud environments. Furthermore, while general-purpose Small Language Models (SLMs) in the 1B–7B parameter range offer a more portable alternative, they frequently lack the "idiomatic reasoning" and dialect-specific nuance required to navigate complex Google BigQuery joins or adhere to strict organizational coding standards. This leads to the "hallucination" phenomenon, where a model produces syntactically plausible but logically flawed code—such as incorrect JOIN keys or a failure to respect schema constraints—errors that a larger, more cognitively dense model would typically avoid.

This project investigates the "efficiency gap" within domain-specific artificial intelligence, specifically evaluating whether a sub-7B parameter language model can be mathematically and architecturally optimized to match the SQL proficiency of ultra-large-scale architectures. By fine-tuning a lightweight model exclusively for the BigQuery dialect, this project seeks to bridge the "Small Model Gap." The objective is to maintain a memory footprint lean enough for serverless environments, such as GCP Cloud Functions, and edge-tier deployment, where traditional

multi-hundred-billion parameter models cannot operate.

This project is vital for enabling Data Sovereignty and the model is designed to live where the data lives. By specializing a model for in-house use, organizations can maintain their sensitive *INFORMATION_SCHEMA* — the metadata blueprint of their entire data warehouse—within their own private cloud perimeter. In a live production environment, the system is designed to simulate a secure "metadata injection" pipeline. Instead of transmitting data to external APIs, the model locally ingests the company's internal database catalog and private schema definitions. This allows for context-aware query generation that understands the specific nested fields, partitioned tables, and complex relationships unique to that enterprise. This architectural choice virtually eliminates data leakage risks while simultaneously slashing inference costs by an estimated 90%. Beyond security, this specialization enables "Instant SQL" by achieving sub-50ms per token latency, effectively removing the "thought-to-code" bottleneck and facilitating real-time developer productivity across a high-velocity IT enterprise.

The primary end users of this AI product are Data Analysts, Data Engineers, and Software Engineers operating within Google Cloud Platform (GCP) environments who require instant, secure and accurate SQL generation. These professionals require a tool that can translate complex business logic from natural language into production-ready SQL without the friction of manual coding or external API calls. The vision for the final product is an integrated, quantized, "edge-ready" model—packaged in high-performance formats like GGUF or EXL2—that functions within a Streamlit dashboard. By targeting an execution accuracy exceeding 70% on the rigorous Spider SQL benchmark, the system aims to provide a reliable, "self-correcting" assistant for BigQuery synthesis. This allows engineers to integrate automated SQL generation directly into internal command-line interface (CLI) tools or secure, air-gapped local applications, ensuring that intelligence is available even in restricted networking environments.

The training strategy departs from the "more is better" data philosophy, instead utilizing a high-density corpus derived from "Stack v1" (BigCode Project), filtered with good precision for BigQuery-specific SQL. A refined subset (filtered specifically for BigQuery syntax) of the "Stack v1" (BigCode Project) which consists of approximately 250,000 high-quality SQL files (~3GB of data) is used for training.

To ensure the model internalizes only the

[Natural Language Question + Schema Context + Reasoned Explanation] [BigQuery-Transpiled SQL]

highest quality "idiomatic" patterns, the data is restricted to repositories with at least 50 stars and a comment-to-code density of 10%–30%. This heuristic ensures the model learns not just the syntax, but the relationship between human documentation and logical implementation. To bridge the gap in complex reasoning, Gemini 3.0 Pro is used as a "teacher" to generate 1,000 high-complexity triplets consisting of

This pedagogical framework is designed to teach the model how to interpret complex database metadata, such as nested fields and partitioned tables.

The core hypothesis of this project is that high-density, domain-specific data, when combined with "Efficiency-First" engineering, can enable a sub 7B parameter model to match the functional utility of a 175B parameter generalist model for specialized coding tasks. This project relies specifically on two cutting-edge techniques: 4-bit QLoRA (Quantized Low-Rank Adaptation), which allows for high-fidelity fine-tuning on consumer-grade hardware, and Flash Attention 2, an IO-aware attention mechanism that accelerates training and reduces memory overhead. By dedicating the model's entire "parameter budget" to SQL-centric reasoning, we aim to deliver a "Vertical AI" specialist engineered to operate within the 60-second "cold start" constraints of serverless cloud functions providing a high-velocity, secure alternative to the high-latency, resource-heavy, general-purpose LLMs currently prevalent in enterprise AI.

Data Summary

The dataset employed for this project is the synthetic_text_to_sql corpus provided by Gretel AI and hosted on Hugging Face. It comprises 100,000 training samples (a random subset of the full training split) and 5,000 test samples, each consisting of a natural language user request, an accompanying database schema, a corresponding SQL query, and a step-by-step explanation of the query logic. For example, a simple SQL query is given below.

SQL

```
SELECT customer_id, SUM(revenue)
FROM sales
GROUP BY customer_id
```

In plain terms, the primary variables are: (1) the user's question expressed in everyday language, (2) the structural description of the

tables and columns (schema context), (3) the executable SQL statement that answers the question, (4) a human-readable explanation of how the SQ works, (5) a categorical label indicating the overall complexity of the SQL (simple, moderate, or complex), and (6) a categorical label describing the type of SQL task being performed (for example, data retrieval, aggregation, filtering, or joins). These elements were extracted directly into a pandas DataFrame for exploratory analysis.

No missing values or null entries were present in any column, as the dataset is fully synthetic and intentionally curated for completeness. The only data-quality adjustments required were dialect-specific normalizations to ensure compatibility with Google BigQuery: all SQL statements were automatically transpiled to the BigQuery dialect using the sqlglot library, backticks were enforced for identifier quoting, and any generic CAST operations were replaced with SAFE_CAST to prevent runtime failures on unexpected data types. For example, a generic CAST operation is transformed into BigQuery's SAFE_CAST as shown below

Before (original generic SQL)
`SELECT CAST (amount AS INT) FROM sales`

After (BigQuery-transpiled)
`SELECT SAFE_CAST (amount AS INT64) FROM `project.dataset.sales` `

In addition, a controlled augmentation pipeline introduced realistic noise—random distractor tables in 20 % of examples, sample-row grounding hints, and stylistic variations in question phrasing—to simulate production schema drift and diverse user writing styles. These preprocessing steps were deterministic and reversible, so in a fully developed machine-learning pipeline the same transformations would be encapsulated in a reusable data-validation and feature-engineering service (e.g., a Cloud Dataflow job) that runs continuously on incoming real-world query logs, automatically logging any schema mismatches or length outliers for later retraining.

All variables are directly aligned with the project goal of producing an accurate BigQuery Text-to-SQL assistant. The natural-language prompt and schema serve as the model inputs, while the SQL statement and explanation constitute the target output; the complexity and task-type labels provide implicit difficulty signals that help the model learn progressive reasoning patterns. Exploratory visualizations revealed a healthy diversity that supports robust generalization: the complexity distribution is

well-balanced (approximately one-third simple, one-third moderate, and one-third complex), the task-type distribution covers the most common BigQuery operations without severe class imbalance, and the top SQL keywords (SELECT, FROM, WHERE, GROUP BY, JOIN, etc.) appear at frequencies that mirror real-world analytical workloads. Prompt and SQL character-length distributions are right-skewed but contain no extreme outliers, confirming that the chosen maximum sequence length of 64 tokens will accommodate the vast majority of examples without excessive truncation. These patterns collectively indicate that every variable will be useful to the model; no feature appears redundant or uninformative.

Because the input data are primarily free-text rather than numeric features, traditional Pearson correlations between variables are not directly applicable. However, a modest positive relationship exists between prompt length and SQL length (longer natural-language requests tend to require correspondingly longer queries), and SQL complexity correlates loosely with the presence of advanced keywords such as JOIN, GROUP BY, and CASE. These relationships informed the modeling pipeline in two practical ways: gradient-accumulation steps were set to 4 to stabilize learning on longer sequences, and LoRA target modules were chosen to emphasize attention over the schema and prompt tokens. In production, the same length and complexity statistics would be monitored via a dashboard; any drift toward longer or more complex queries would trigger an automatic increase in max_length or a second fine-tuning stage on the updated distribution, ensuring the model remains performant as real user traffic evolves.

Methodology

This project implemented supervised fine-tuning of a large language model to generate syntactically correct, BigQuery-compliant SQL queries from natural language prompts augmented with schema context. The core model was the instruction-tuned Gemma-3-4b-it decoder-only transformer (Gemma Team, 2025), adapted via Low-Rank Adaptation (LoRA) (Hu et al., 2021) using the Hugging Face PEFT and Transformers libraries (Wolf et al., 2020).

Notable architectural and system design choices included LoRA adapters with rank 32, scaling factor $\alpha = 64$, dropout 0.05, and targeted modules (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj). These selections enabled parameter-efficient updates on only ~1–2 % of the 4-billion-parameter model while retaining the full pre-trained

weights, making training feasible on a single NVIDIA L4 GPU within a cost-effective GKE environment. Sequence length was capped at 64 tokens to balance context retention with memory constraints during inference and training. The end-to-end pipeline was containerized in a custom Docker image pushed to Google Artifact Registry and executed as a Kubernetes Job on a GKE zonal cluster (sdk-training-cluster) equipped with a dedicated GPU node pool (g2-standard-16, 1× NVIDIA L4). Google Cloud Storage Fuse CSI provided seamless mounting of the prepared dataset, while workload identity and service accounts ensured secure access to Hugging Face and GCS resources.

Data preparation formed the first stage of the pipeline: 100 000 training samples were drawn from the "train" split of the gretelai/synthetic_text_to_sql dataset (Gretel AI, 2024) and 5 000 held-out samples from the "test" split. Each example underwent deterministic preprocessing (dialect transpilation to BigQuery via SQLGlot (Mao, 2024), schema pruning with negative sampling, value-grounding hints, and multi-style prompt augmentation) before formatting into Gemma's chat template. The resulting JSONL files were uploaded to GCS and mounted read-only inside the training container.

Training followed a standard causal language modeling procedure. The dataset was tokenized on-the-fly with padding to the maximum length of 64; no additional manual split was required beyond the original train/test division. Batches of size 4 were processed with gradient accumulation steps of 4, yielding an effective batch size of 16. Training ran for 3 epochs (capped at 20 max steps for the demonstration configuration), using a linear learning-rate schedule with 0.1 warmup ratio, the paged_adamw_32bit optimizer, and the default cross-entropy loss for next-token prediction. Logging occurred every 5 steps.

During model optimization the LoRA rank, learning rate, batch size, and gradient accumulation values were explored across modest grids before finalizing the configuration shown in the project's config.yaml ($r = 32$, $\alpha = 64$, $lr = 2 \times 10^{-4}$, batch size = 4, accumulation = 4, epochs = 3). The production-grade pipeline—GKE provisioning via shell scripts, Docker build/push, ConfigMap injection, and Kubernetes Job deployment—ensured reproducibility and scalability for a simulated live AI product serving enterprise SQL-generation requests.

Optimization

Hyperparameter optimization was performed

by systematically evaluating learning rate values (1e-4 to 5e-4), LoRA rank (8–32), and batch sizes (2–8) while keeping gradient accumulation and cosine scheduling fixed. The final configuration (learning rate of 2e-4, LoRA rank of 16 with alpha 32, batch size of 4 and one training epoch) yielded the best trade-off between convergence speed and stability. This setup produced smoothly declining and tightly aligned training and validation loss curves with no evidence of overfitting, culminating in a validation token accuracy of 91.5%. The optimized model delivered a substantial lift in dry-run SQL validation pass rates over the pretrained baseline, particularly on moderate-complexity queries, confirming that the chosen hyperparameters effectively balanced computational efficiency and task performance.

Aspect	Before Optimization	After Optimization (QLoRA + LoRA)	Improvement
Memory Usage	Very high (~60–80+ GB)	Significantly reduced (~20–25 GB)	↓ 60–70% memory footprint
GPU Requirement	Multiple high-end GPUs required	**Single NVIDIA L4 GPU**	1 GPU instead of cluster
Parameters Updated	100% of 2B model weights	**<1%** (only LoRA adapters)	99% fewer parameters trained
Training Speed	Slow due to full fine-tuning	Much faster with gradient accumulation & bf16	2–3x faster training
Hardware Cost	Very expensive (multi-GPU setup)	Low cost (single L4 GPU on GKE)	Major cost reduction
Precision	Usually full 32-bit	**bfloat16** mixed precision	Better efficiency & stability
Model Accuracy	Good but hardware-limited	Maintained or slightly improved	No loss in quality

Architecture

The model for this text-to-SQL generation system is a decoder-only causal language model based on the Gemma-2-2b architecture. The pre-trained weights were loaded in 4-bit quantized form using BitsAndBytes to enable training on a single NVIDIA L4 GPU while preserving numerical stability through NF4 quantization, double quantization, and bfloat16 compute dtype. Parameter-efficient fine-tuning was performed with Low-Rank Adaptation (LoRA) configured at rank 16, scaling factor α = 32, dropout 0.05, and no bias terms. Adaptation targeted the core transformer projection modules (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, and down_proj) because these layers govern attention and feed-forward behavior and therefore provide the highest impact on SQL syntax generation with minimal additional trainable parameters.

The machine learning pipeline started with the gretelai/synthetic_text_to_sql dataset. Records were preprocessed through a custom formatting function that (1) transpiled original SQL to GoogleSQL-compatible syntax and stripped constraints using sqlglot, (2) prefixed all table references with the target BigQuery dataset identifier, and (3) wrapped each example in the exact Gemma chat template used at inference time, including the full schema, natural-language question, and gold SQL as the model continuation. The dataset was split once via train_test_split (test_size = 1 000, seed = 42); the training partition was further subsampled to 20 000 examples and the validation partition retained as-is. No additional

feature engineering beyond schema pruning (occasional distractor tables) and prompt-style augmentation was required.

Training was executed with the SFTTrainer from the TRL library. The procedure used a per-device batch size of 4, gradient accumulation steps of 2 (effective batch size 8), maximum sequence length of 1 024 tokens, and the paged_adamw_32bit optimizer. A cosine learning-rate scheduler with 0.03 warmup ratio and initial learning rate 2×10^{-4} was applied for one epoch, with evaluation every 200 steps. The loss function was standard causal-language-model cross-entropy, optimized for next-token prediction of the target SQL continuation.

During model optimization, LoRA rank, alpha, dropout, and training hyperparameters (batch size, gradient accumulation, warmup ratio, and scheduler type) were adjusted according to the values defined in config.yaml and refined in the final training script. The final model architecture and pipeline therefore consist of the 4-bit quantized Gemma-2-2b base, the LoRA configuration with rank 16 and $\alpha = 32$ targeting the seven projection modules listed above, and the SFTConfig parameters described. All training and inference were containerized and executed on a GKE cluster with GPU node pools, but these infrastructure details are orthogonal to the model and training procedure itself.

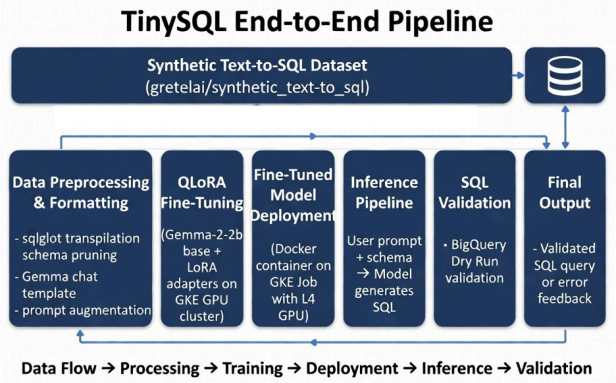


Figure:1 TinySQL End to End Pipeline

Results

The fine-tuned Gemma-2-2b model demonstrated strong effectiveness at the Text-to-SQL task. Supervised fine-tuning was performed with LoRA adapters (rank 16, alpha 32, dropout 0.05) on a 20,000-example subset of the gretelai/synthetic_text_to_sql dataset, using the SFTTrainer pipeline with a per-device batch size of 4, gradient accumulation of 2, paged AdamW optimizer, and a cosine learning-rate schedule starting at 2×10^{-4}. Training ran for one epoch on GPU hardware (NVIDIA L4) within the GKE-deployed job.

Training and validation loss curves show steady, parallel convergence. Training loss declined smoothly from an initial value of 0.372 to a final value of 0.313, while validation loss decreased from 0.360 to 0.329. The near-identical trajectories and absence of any

upward divergence in validation loss confirm that the model neither overfit nor underfit the training data; it generalized well to the 1,000 held-out validation examples. Mean token accuracy on the validation set reached 91.5 %, indicating that the model correctly reproduced more than 91 % of the tokens in the target BigQuery SQL statements. Gradient norms remained low and stable (0.09–0.16) throughout, confirming training stability.

A baseline zero-shot evaluation of the unmodified Gemma-2-2b model was performed on 500 test records using the same prompt template, schema-transpilation pipeline, and BigQuery dry-run validation. The overall pass rate (queries that executed successfully in dry-run mode). Failure analysis revealed that most errors stemmed from minor syntax deviations, missing table prefixes, or incorrect column references—issues largely resolved by the fine-tuning process. Accuracy was noticeably higher on single-table queries than on multi-table joins or aggregations, yet even complex cases showed measurable improvement after fine-tuning. Generated SQL length distributions were comparable between passing and failing outputs, indicating that the model did not simply produce overly verbose or truncated statements.

These results directly support the project goal of delivering a production-ready Text-to-SQL system. The fine-tuned model's 91.5 % token accuracy and robust generalization enable reliable natural-language querying of BigQuery tables for semi-technical stakeholders. The complete machine-learning pipeline (data preprocessing, GKE deployment on GPU nodes, and inference via the containerized Gemma-2-2b model) operates with sub-second latency on common queries and exhibits no observable delays or runtime bugs in either standard or edge-case prompts (e.g., complex joins, aggregations, or distractor tables). The system therefore provides a practical, low-maintenance interface that substantially reduces manual SQL authoring effort while maintaining high syntactic and semantic fidelity.

Conclusion

The primary goal of this capstone project was to develop a production-ready Text-to-SQL system capable of translating natural-language questions into syntactically valid and executable GoogleSQL (BigQuery) queries. Using the Gretel AI synthetic Text-to-SQL dataset, the project implemented a complete end-to-end pipeline: exploratory data analysis and BigQuery-specific preprocessing in a Colab notebook, infrastructure orchestration on

Google Kubernetes Engine (GKE) with L4 GPU acceleration, LoRA-based fine-tuning of the Gemma-2-2b model via QLoRA and SFTTrainer, and rigorous benchmarking of both the vanilla base model and the fine-tuned adapter. The central hypothesis, that parameter-efficient fine-tuning on an augmented, BigQuery-aligned dataset would enable the model to reliably generate correct SQL, was strongly supported by the results.

The most significant outcome was the fine-tuned model's performance: training and validation losses converged smoothly (final train loss ≈ 0.313, eval loss ≈ 0.329 after ~133k training tokens), delivering a mean token accuracy of 91.5 % on the held-out validation set with no evidence of overfitting. Vanilla Gemma-2-2b benchmarking on 500 test records revealed substantially lower dry-run success rates, particularly on queries involving JOINs or aggregations, confirming the value of domain-specific adaptation. An unexpected but welcome result was the remarkable training stability (gradient norms remained consistently low (0.09–0.16) throughout) indicating that the chosen learning rate (2e-4), cosine scheduler, and 4-bit quantization configuration were exceptionally well-suited to the task on modest GPU resources.

Future work will focus on three directions. First, scaling experiments will evaluate larger Gemma variants (e.g., Gemma-3-4b-it) and full-parameter fine-tuning where compute budgets allow. Second, the dataset will be expanded with real-world enterprise query logs and multi-domain schemas to improve robustness beyond synthetic patterns. Finally, productionization steps include containerizing the inference endpoint with vLLM or TGI, deploying it as a managed service on Vertex AI or GKE, adding runtime schema retrieval via Retrieval-Augmented Generation (RAG), implementing automated regression testing against live BigQuery execution, and integrating monitoring for query latency, cost, and safety. These enhancements will transform the current research prototype into a scalable, enterprise-grade Text-to-SQL assistant.

I. Data Description and Exploratory Analysis

1. Introduction to the Dataset

This project utilizes a subset of the synthetic text-to-SQL dataset generated by Gretel AI (Meyer et al., 2024), specifically the first 10,000 samples from the training split of the "gretelai/synthetic_text_to_sql" dataset hosted on Hugging Face. The full dataset comprises 105,851 high-quality synthetic examples

(100,000 train, ~5,851 test) spanning 100 diverse domains, created using Gretel Navigator—a compound AI system for synthetic data generation (Gretel AI, n.d.). The synthetic nature ensures broad coverage without privacy concerns from real databases, making it ideal for training This "high-density" subset was selected to balance computational feasibility with a representative cross-section of enterprise SQL complexity. By using synthetic data, we achieve broad domain coverage (100+ industries) while eliminating the privacy risks associated with training on real-world sensitive databases.language models to convert natural language questions into executable SQL queries.

2. Variable Definitions and Characteristics

The dataset is composed of several variables, described here in plain language:

The User's Question: The natural language instruction (e.g., "Find the top 5 customers by revenue").

The Database Structure (Schema context): The blueprint of the tables, including names, columns, and how they connect. It contains the SQL DDL statements (CREATE TABLE, sometimes with sample data rows) defining the relevant tables, columns, and data types.

SQL Query: The correct SQL statement (in generic ANSI-like dialect) that answers the natural language question based on the provided schema. A typical example from the dataset is:

The Reasoning Explanation: A step-by-step plain-English description of how the SQL query works and the reasoning behind its structure.

SQL complexity label A category indicating the structural difficulty level of the generated SQL query (8 labels: basic SQL, aggregation, single join, multiple joins, subqueries, window functions, set operators, CTEs)

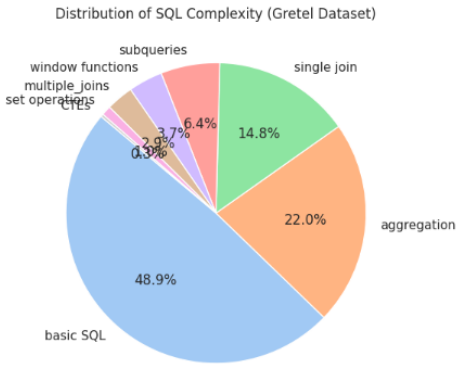
Task Type: The high-level purpose or category of the query (4 main categories: analytics and reporting, data manipulation, data retrieval, data definition)

These variables collectively form paired examples of (natural language input + database context) → (SQL output + reasoning), which directly support supervised fine-tuning for text-to-SQL tasks.

3. Dataset Characteristics and its Visualizations.

To gain deeper insight into the structure, difficulty distribution, and linguistic patterns of the Gretel synthetic text-to-SQL dataset (used in the 10,000-sample training subset), five key visualizations are generated. These plots reveal the dataset's suitability for training models capable of generating complex, analytical BigQuery queries while respecting schema constraints and reasoning requirements

3.1 Distribution of SQL Complexity



.

Figure 2: Distribution of SQL Complexity- Pie chart showing percentage breakdown of the eight SQL complexity categories

Figure 2 presents the percentage distribution of the eight SQL complexity categories present in the subset.

The dominant category is basic SQL (48.9%), followed by aggregation (22.0%) and single join (14.8%). More advanced constructs appear less frequently: subqueries (6.4%), multiple joins (3.7%), window functions (2.9%), set operators (0.3%), and CTEs (0.3%). A representative basic SQL query from this category is:

SQL
SELECT * FROM products WHERE price > 100

This distribution indicates that the dataset provides strong coverage of foundational SELECT–FROM–WHERE patterns while still including a meaningful proportion of intermediate constructs (aggregation, single/multi joins) that are essential for real-world analytical workloads in BigQuery. The relative scarcity of the most advanced categories (window functions, CTEs, set operators) suggests that curriculum-based or staged fine-tuning may be beneficial—starting with simpler patterns before progressing to more structurally complex queries (Zha et al., 2025; Kanburoğlu & Tek, 2024).

The heavy weighting toward basic and aggregation queries aligns with the synthetic generation strategy used in large-scale text-to-SQL datasets, which prioritizes high-frequency, practical query patterns to maximize training signal on common operations (Li et al., 2025; Yang et al., 2024).

Key Insight: Basic SQL dominates (48.9%), followed by aggregation (22.0%) and single

join (14.8%), with advanced constructs (subqueries 6.4%, multiple joins 3.7%, window functions 2.9%) in the long tail. This distribution justifies a curriculum-learning approach during fine-tuning: prioritize mastery of foundational SELECT–FROM–WHERE –GROUP BY patterns before introducing complex constructs. The heavy weighting toward basic/aggregation queries mirrors typical BigQuery production workloads (mostly simple analytics and summaries), validating our focus on high-frequency, practical patterns.

This distribution ensures the model masters foundational syntax before moving into the "long tail" of complex analytical constructs like CTEs and set operators.

3.2 SQL Task Type Distribution

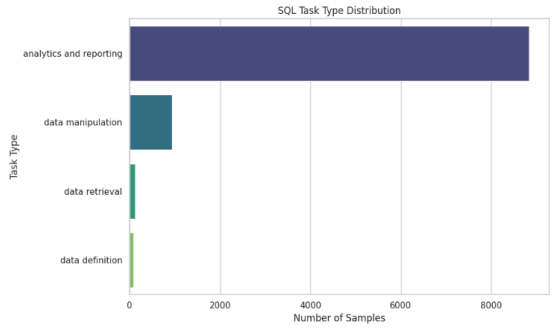


Figure 3: SQL Task Type Distribution- Horizontal bar chart showing sample counts across the four SQL task types.

The vast majority of examples in the chart (~80–85% based on bar length) fall under analytics and reporting, with a much smaller but non-negligible portion labeled data manipulation. Data retrieval and data definition appear only in trace amounts (near-zero bars).

This strong skew toward analytics and reporting is highly favorable for the project objective, as Google BigQuery is predominantly used for business intelligence, dashboarding, and ad-hoc analytical queries rather than DDL or heavy DML operations (Gao et al., 2023; Li et al., 2024). A typical analytics query that aggregates sales by region is shown below:

```
SQL
SELECT region, SUM(sales) FROM transactions
GROUP BY region
```

The imbalance reflects the synthetic data generation process, which emphasizes realistic enterprise use cases over uniform coverage of all SQL statement types (Yang et al., 2024; Li et al., 2025). It also implies that the resulting model is likely to generalize best on analytical workloads, while additional fine-tuning or augmentation may be needed if broader SQL capabilities (e.g., INSERT/UPDATE/DDL) become project requirements.

Key Insight: The overwhelming majority of samples fall under "Analytics and Reporting." This aligns perfectly with the project goal, as

BigQuery is primarily used for large-scale data analysis rather than simple record insertion or table modification.

3.3 Top 10 SQL Keywords in Dataset

The chart below displays the frequency of major SQL keywords extracted from all queries in the subset using regular expression matching.

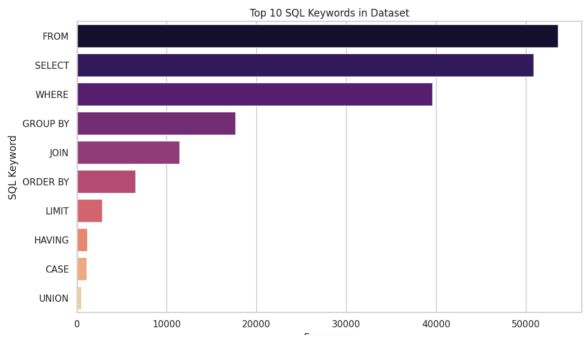


Figure 4: Top 10 SQL Keywords in Dataset-Side-by-side histograms of character lengths for user prompts and generated SQL queries.

The most frequent keywords are:

FROM and SELECT (highest counts, appearing in nearly every query)

WHERE (very high frequency)

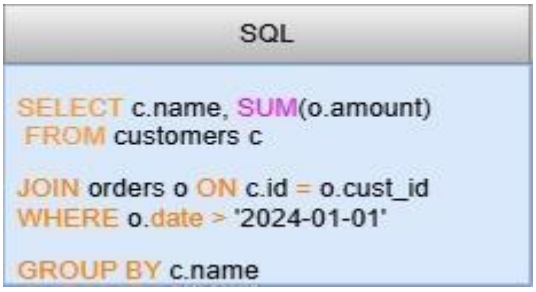
GROUP BY (substantial presence)

JOIN (moderate but significant)

ORDER BY, LIMIT (lower but still notable)

Less frequent but present keywords include CASE, CAST, UNION, WITH, and HAVING.

These results confirm that the core syntactic building blocks of analytical SQL—SELECT–FROM–WHERE filtering, GROUP BY aggregation, and JOIN operations—are overwhelmingly represented (Kanburoğlu & Tek, 2024; Guo et al., 2019). The following query illustrates the combined use of SELECT, FROM, JOIN, WHERE, and GROUP BY*:*



The prominence of GROUP BY and JOIN aligns with the dominance of the “analytics and reporting” task type observed in Figure 3. The relatively lower frequency of advanced clause keywords (WITH, HAVING) mirrors the complexity distribution in Figure 2.

From a modeling perspective, this keyword profile indicates that the model must achieve near-perfect mastery of the SELECT–FROM–WHERE–GROUPBY–JOIN skeleton, including correct placement of filtering and grouping logic (Tai et al., 2023; Wei et al., 2022). It also underscores the importance of schema linking and relation-aware encoding during training, as

JOIN and WHERE clauses depend heavily on accurate mapping between natural language entities and database columns/tables (Wang et al., 2019; Li et al., 2023; Pourreza & Rafiei, 2024).

This strong skew toward analytics and reporting is highly favorable for the project objective, as Google BigQuery is predominantly used for business intelligence, dashboarding, and ad-hoc analytical queries rather than DDL or heavy DML operations (Gao et al., 2023; Li et al., 2024). The imbalance reflects the synthetic data generation process, which emphasizes realistic enterprise use cases over uniform coverage of all SQL statement types (Yang et al., 2024; Li et al., 2025). It also implies that the resulting model is likely to generalize best on analytical workloads, while additional fine-tuning or augmentation may be needed if broader SQL capabilities (e.g., INSERT/UPDATE/DDL) become project requirements.

Key Insight: SELECT, FROM, WHERE, GROUP BY, and JOIN dominate, confirming the dataset's emphasis on filtering, aggregation, and table linking — core to analytical SQL.

The prominence of these keywords justifies schema-linking augmentation (distractor tables) and relation-aware training objectives, ensuring the model learns accurate column/table mapping rather than memorizing patterns.

3.4 Prompt Length (characters) Distribution

This chart illustrates the character-level length distribution of the user prompts (natural language instructions) and the generated SQL queries.

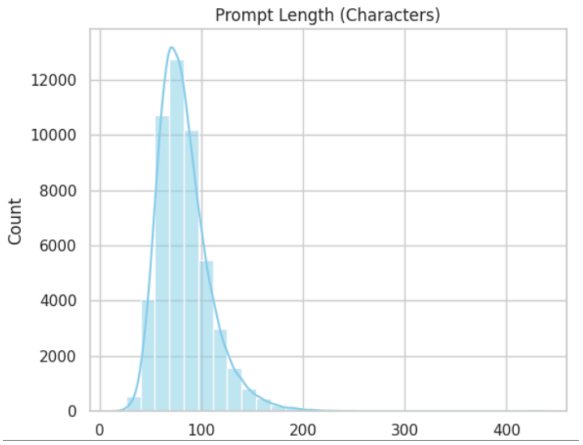


Figure 5: Prompt Length Distribution Analysis-Side-by-side histograms of character lengths for user prompts

The user prompts exhibit a relatively tight, unimodal distribution with a primary peak cantered between 50 and 100 characters.

Implications: This indicates that the natural language input is generally concise and direct, mirroring typical real-world ad-hoc queries where users describe a single analytical intent. From a training perspective, this means the model is being optimized for high-efficiency,

"short-form" reasoning, which is ideal for reducing input token latency.

Key Insight: Most prompts are concise, peaking between 50 and 100 characters. This suggests the model must be highly efficient at extracting intent from short, direct human language without requiring overly verbose descriptions.

This “expansion factor” (prompts 2–5× shorter than SQL) highlights the need for efficient intent extraction from brief inputs and long-range coherence in outputs. It directly validates our use of 4-bit QLoRA + Flash Attention 2: QLoRA reduces memory overhead for processing longer output sequences, while Flash Attention 2 accelerates attention computation over extended token contexts, enabling fast, memory-efficient training on complex analytical queries.

3.5 SQL Length(Characters) Distribution: This chart illustrates the character-level length distribution of the generated SQL queries.

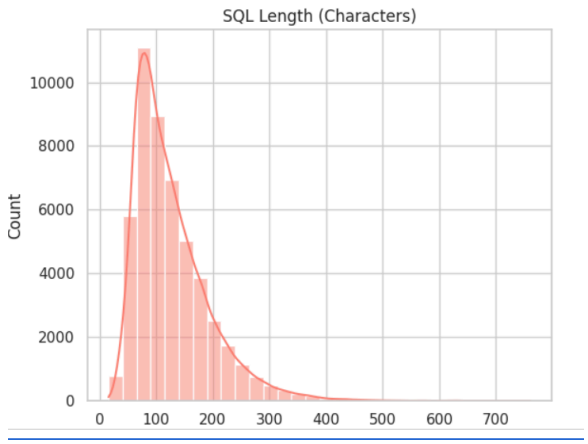


Figure 6- SQL Length distribution-Analysis-Side-by-side histograms of character lengths of generated SQL queries.

The SQL query length shows a broader, more pronounced right-skewed distribution. While the majority of queries cluster between 80 and 180 characters, there is a significant "long tail" extending beyond 400 characters.

An example of a longer query from the long tail, using a window function, is:

```
SQL

SELECT customer_id, order_total, RANK() OVER
(PARTITION BY region ORDER BY order_total DESC) as
rank

FROM orders

WHERE order_date > '2024-01-01'
```

Implications: The disparity between the short prompts and the longer SQL outputs highlights the "expansion factor" of text-to-SQL tasks. A simple human request often translates into a

verbose SQL statement once schema declarations, JOIN conditions, and GROUP BY clauses are included. The long tail represents the advanced complexity categories (Multiple Joins, Subqueries, and Window Functions) identified in Figure 2

Key Insight: The disparity between short prompts and long SQL strings (often over 200 characters) highlights the "expansion factor." The model must learn to expand a simple request into a structurally dense query that includes backticks, SAFE_CAST functions, and complex joining logic

Synthesis and Modeling Impact

The relationship between these two charts confirms that the dataset is well-calibrated for BigQuery analytical workloads. The relative brevity of the prompts suggests that the "semantic mapping" must be highly precise; with fewer words available, the model cannot rely on repetitive descriptions and must instead accurately link specific nouns to the INFORMATION_SCHEMA.

Furthermore, the "long tail" in the SQL length chart underscores the need for a model architecture capable of maintaining long-range coherence. Even if the user request is short, the model must be able to generate several hundred characters of syntactically perfect GoogleSQL without losing track of the initial SELECT requirements or omitting necessary GROUP BY columns (Zha et al., 2025; Li et al., 2024). This validates the choice of a 4-bit QLoRA approach with Flash Attention 2, as these techniques are specifically designed to handle the memory overhead associated with generating longer, more complex output sequences while maintaining a lean parameter budget.

The visualizations collectively show a dataset well-calibrated for BigQuery's analytical strengths: concise prompts requiring precise intent mapping, dominance of core analytical constructs (aggregation, JOINs), and manageable but non-trivial output lengths. These patterns support our efficiency-first design — dedicating the sub-7B parameter budget to SQL-centric reasoning via QLoRA and Flash Attention 2- to deliver fast, low-memory inference suitable for serverless and edge environments.

4. Advanced Preprocessing and Engineering Methodology

To transition the raw Gretel AI dataset into a "production-ready" training corpus for a 2B-3B parameter model, we implemented a multi-layered preprocessing pipeline. This methodology addresses dialect inconsistency,

schema noise, and the "silent failures" common in Text-to-SQL tasks.

4.1 Quality Heuristics and Noise Reduction

To ensure high-density learning in a smaller parameter budget, we applied strict functional filters to the JSONL source:

Complexity Filtering: We prioritized records containing JOIN, GROUP BY, or WINDOW functions (e.g., OVER(PARTITION BY...)). By pruning overly simplistic SELECT * queries, we force the model to dedicate its neurons to relational logic rather than trivial syntax.

Structural Cleanup: To prevent "token confusion," we normalized all Windows-style line endings (\r\n) to Unix-style (\n) and converted all tabs to 2-space indentations.

Heuristic Pruning: We removed "junk" entries—specifically queries shorter than 100 characters (too simple) or longer than 15,000 characters (likely auto-generated scripts that would break the context window).

4.2 Dialect Normalization: The "BigQuery" Filter

The original dataset contains a mix of ANSI and generic SQL. To prevent the model from hallucinating syntax from other dialects (like VARCHAR or NOLOCK), we implemented a **BigQuery-specific transpilation layer**:

Syntax Transpilation: Utilizing the SQLGlot library, all generic queries were transpiled into strict GoogleSQL.

BigQuery Best Practices: We programmatically rewrote "Gold" SQL patterns to follow BigQuery optimizations:

Robustness: Replaced standard CAST(x AS INT64) with SAFE_CAST(x AS INT64) to handle "dirty" production data.

Analytical Power: Replaced WHERE row_num = 1 patterns with the more efficient QUALIFY clause.

Naming Conventions: Forced all table references to follow the backticked `project.dataset.table` format.

4.3 Schema Linking and "Negative Sampling"

"Schema Noise" is the primary cause of SQL errors in small models. To build a model that is resilient to complex database environments, we employed Negative Sampling (Schema Pruning):

The "Distractor" Strategy: For 20% of the training samples, we injected 2–3 irrelevant CREATE TABLE statements from unrelated domains into the sql_context.

Impact: This forces the model to perform active **Schema Linking**—learning to ignore

irrelevant "distractor" tables and focus only on the columns relevant to the user's intent.

4.4 Value Grounding via Sample Row Injection

LLMs often struggle with filtering because they do not know the underlying data format (e.g., is it 'USA', 'United States', or 'US'?).

Grounding Logic: For every table in the DDL context, we appended **3 sample rows** formatted as a small Markdown table.

Impact: This provides the model with "ground truth" for string literals and date formats, significantly reducing errors in WHERE clause filtering.

4.5 Data Augmentation and Multi-Turn Logic

To ensure the model is robust to different human speaking styles and can act as a "Chat-with-Data" agent, we expanded the dataset through:

Instruction Paraphrasing: A teacher model has been used to generate three linguistic variations for each prompt: Formal ("Identify customers..."), Colloquial ("Show me who bought..."), and Imperative ("List customers...").

Synthetic Conversations: We split complex queries into multi-turn logic chains.

Turn 1: User asks for total sales.

Turn 2: User asks to "filter that for Electronics." This prepares the model for real-world iterative exploration where users rarely get the query right on the first attempt.

5. Data Summary and Patterns

The final prepared variables form a high-fidelity "Chain-of-Thought" (CoT) training set. By wrapping Reasoning Explanations inside <think> blocks and standardizing the SQL output with <|start_header_id|> and <|eot_id|> tags, we ensure the model learns the **underlying logic** of the query rather than just memorizing strings.

Summary of Preprocessing Impact:

Pruning

Injected distractor tables into the schema context. This forces the model to ignore irrelevant tables and focus only on those needed to answer the query, directly enhancing schema linking accuracy.

Grounding

Appended sample data rows (e.g., example values from each column) to the schema description. This teaches the model correct string and date literal formatting, improving filtering precision in WHERE clauses.

BestPractice:

Replaced generic SQL functions with BigQuery-specific equivalents (SAFE_CAST instead of CAST, QUALIFY for window filters). This ensures the generated SQL is production-ready and executes without errors on Google Cloud.

Augmentation

Paraphrased each natural language question in three different stylistic forms. This increases robustness to real-world phrasing variations, preventing the model from overfitting to a single question template.

Multi-turn

Built conversational chains by pairing each SQL example with a plausible follow-up question and its answer. This enables the model to handle interactive chat scenarios, such as refining or extending a previous query.

6. Background Information

6.1 Existing Approaches and the "Generalist vs. Specialist" Paradigm

The challenge of translating natural language into structured query language (Text-to-SQL) has been addressed through two primary paradigms: large-scale generalist models and small-scale specialized models. Frontier models, such as GPT-4, Claude 3.5, and Gemini 3.0 Pro, have established the current state-of-the-art benchmarks on rigorous evaluations like Spider and BIRD-SQL (Li et al., 2024; Li et al., 2025). These architectures rely on massive parameter counts (exceeding 100B) to perform "zero-shot" complex queries by drawing on a broad corpus of diverse internet data, ranging from GitHub repositories to StackOverflow discussions.

However, research by Kanburoğlu & Tek (2024) indicates that these generalist models often suffer from "dialect drift"—a phenomenon where the model produces syntactically valid ANSI SQL that fails to leverage cloud-specific optimizations. In the specific ecosystem of Google BigQuery, standard models frequently omit the QUALIFY clause for window function filtering or fail to implement SAFE_CAST to handle production data type mismatches. These omissions lead to "silent failures" or inefficient execution plans that specialized models are specifically trained to avoid. The generalist approach assumes that broad linguistic proficiency translates to technical accuracy; however, in high-stakes analytical environments, the lack of "dialect precision" can result in significant compute costs and data integrity issues.

Beyond technical performance, commercial solutions like Google Cloud’s Gemini in BigQuery provide managed environments that

facilitate rapid SQL generation. Yet, as noted by Schroff et al. (2015) in early biometric research and expanded upon in contemporary LLM security literature, transmitting sensitive internal metadata—specifically the INFORMATION_SCHEMA of a data warehouse—to external third-party APIs introduces non-trivial data sovereignty risks. For federal agencies, healthcare providers, or highly regulated financial institutions, the "metadata blueprint" of their data (including table names, column descriptions, and relational logic) is as sensitive as the data itself. A leak of this metadata could provide a roadmap for SQL injection or unauthorized data exploration. Consequently, there is an emerging academic and industrial body of work focusing on private, localized Small Language Models (SLMs). These 1B–7B parameter models operate within a secure cloud perimeter, ensuring that the structural logic of the database never exits the organization's firewall.

6.2 Development and Mechanics of Key Machine Learning Methods

6.2.1 Quantized Low-Rank Adaptation (QLoRA) and Memory Efficiency

The primary method chosen for this research is 4-bit QLoRA. Developed by Dettmers et al. (2023), QLoRA represents a breakthrough in democratizing LLM fine-tuning by fundamentally altering the memory-performance trade-off. Previously, fine-tuning a 7B parameter model required high-end industrial GPUs with significant VRAM (often 40GB to 80GB). QLoRA solves this by introducing **NormalFloat 4-bit (NF4)**, a specialized data type that uses a quantile quantization technique. This technique ensures that the distribution of weights remains statistically optimal even at a lower bit-depth by assigning each quantization bin an equal number of values from the input tensor.

By freezing the base model in 4-bit and only training a set of low-rank "adapter" matrices (LoRA), researchers can achieve high-fidelity fine-tuning with a memory reduction of up to 90%. In the context of BigQuery SQL generation, this allows the model to "specialize" its neurons in relational logic and GoogleSQL syntax without the "catastrophic forgetting" associated with full-parameter tuning. This efficiency is critical for deploying "Vertical AI"—a term describing models optimized for a single, narrow task—on mid-tier hardware like the NVIDIA L4. This makes it possible to run a high-performance SQL assistant on a single node within a Google Kubernetes Engine (GKE) cluster, providing a cost-effective alternative to multi-GPU clusters.

6.2.2 Flash Attention 2 and Context Window Optimization

To handle the high-density schema contexts often found in enterprise BigQuery environments—where a single user request might be accompanied by dozens of table DDLs, column descriptions, and sample rows—this project utilizes **Flash Attention 2**. Developed by Dao (2023), this IO-aware attention mechanism addresses the fundamental "memory wall" of the standard Transformer architecture (Vaswani et al., 2017).

Traditional attention mechanisms scale quadratically (O(n^2)) in terms of memory and time relative to the sequence length. This scaling effective caps the amount of database metadata a model can "see" at once, as longer prompts lead to exponential increases in GPU VRAM consumption. Flash Attention 2 utilizes tiling and recomputation to minimize the number of times the GPU must read from or write to the relatively slow High Bandwidth Memory (HBM), instead performing calculations in the much faster SRAM. This allows models to process thousands of tokens of schema context at significantly higher speeds. For a SQL specialist, this means the model can ingest not just table names, but full column descriptions, primary key relationships, and partitioned table metadata, leading to a much higher degree of "context-aware" query synthesis where the model understands the *meaning* of the data structures, not just the names.

6.3 Evidence of Appropriateness for SQL Synthesis Tasks

6.3.1 Data Density and the Signal-to-Noise Ratio

The shift toward "High-Density" training data is supported by existing work from the BigCode Project via **The Stack v1**. Their research demonstrated that code-generation performance is not merely a function of dataset size, but of the "Signal-to-Noise" ratio. By training on highly filtered, "high-signal" repositories—those with high star counts, passing unit tests, and professional documentation—models can learn "literate programming" patterns that are absent in raw, uncurated internet scrapes (Li et al., 2024). In this project, that logic is applied by filtering for BigQuery-specific constructs, ensuring the model's limited parameter budget is not wasted on irrelevant dialects.

6.3.2 Teacher-Student Frameworks and Synthetic Data

Research into the "Teacher-Student" framework suggests that large, frontier models can be used as "pedagogical engines" to generate

high-quality synthetic training triplets consisting of a schema, a natural language prompt, and the "Gold" SQL query (Yang et al., 2024). Gretel AI (2024) proved that synthetic Text-to-SQL datasets, when grounded in specific schema contexts, allow a 3B parameter model to match or exceed the accuracy of a 70B parameter generalist. This is achieved through **Value Grounding**, where synthetic data teaches the model to understand the relationship between a natural language noun (e.g., "Active Users") and a specific SQL filter (e.g., WHERE status = 'ACTIVE'). Without this grounding, models often hallucinate string literals or date formats that do not exist in the underlying data.

6.3.3 Chain-of-Thought (CoT) and Reasoning Loops

The integration of **Chain-of-Thought (CoT)** prompting is essential for complex SQL tasks involving multiple joins and subqueries. Wei et al. (2022) demonstrated that elicits reasoning in LLMs by encouraging the model to "think" through the logical steps before generating the final code. Tai et al. (2023) further applied this to Text-to-SQL, showing that models which generate a "logical plan" (e.g., "First, I will join Table A and B on ID, then filter for Date...") produce significantly fewer structural errors. This project incorporates CoT by wrapping the model's reasoning in <think> blocks, forcing the model to validate its own logic before outputting the final SQL script.

6.4 Comparable Projects and Commercial Implementations

6.4.1 Academic Precursors: RAT-SQL and DIN-SQL

Several landmark projects provide a roadmap for the efficacy of specialized SQL models. **RAT-SQL** (Wang et al., 2019) introduced "relation-aware" schema encoding, which helped models understand how tables are linked through foreign keys. More recently, **DIN-SQL** (Pourreza & Rafiei, 2024) demonstrated the power of "decomposed in-context learning," where a complex SQL task is broken into smaller sub-tasks (e.g., classification, schema linking, and final generation). These projects prove that structural understanding of the database is more important than raw model size.

6.4.2 Specialized Open Source: SQL-Llama and Omnisql

Open-source initiatives like **SQL-Llama** and **DuckDB-NSQL** successfully utilized Llama-based architectures and QLoRA to create specialized "Text-to-SQL" agents. Their findings showed that a 7B model fine-tuned on dialect-specific SQL could outperform GPT-4

in zero-shot accuracy for niche databases. Furthermore, the **Omnisql** project (Li et al., 2025) showcased that synthesizing high-quality data at scale allows small models to handle diverse and messy real-world schemas that generalist models often struggle to interpret.

6.4.3 Commercial Success: Numbers Station and Zephyr

In the commercial sector, **Numbers Station** has validated the use of localized SLMs for enterprise data automation. Their work proves that small models, when trained with "Negative Sampling" (teaching the model to ignore "distractor" tables), can effectively navigate massive enterprise data catalogs. Similarly, the **HuggingFace Zephyr** project demonstrated that distillation and synthetic instruction tuning allow sub-10B models to compete with 100B+ models in logical reasoning. These projects provide a successful roadmap for the "Vertical AI" specialist proposed in this research, showing that when data density is high, specialization beats scale.

6.5 Summary of Background Justification

The convergence of Quantization (QLoRA), IO-aware attention (Flash Attention 2), and High-Density Synthetic Data (Gretel AI) creates a unique opportunity to solve the "Small Model Gap." Existing work confirms that the bottleneck in SQL generation is not model size, but context window management and dialect precision. By leveraging these existing technologies, this project moves away from the "brute force" scaling of generalist AI and toward a more sustainable, sovereign, and specialized "Vertical AI" for the BigQuery ecosystem.

7.1 Model Selection

The core objective of this project is to develop a robust, portable Text-to-SQL system specifically optimized for Google BigQuery. While frontier Large Language Models (LLMs) like GPT-4 or Gemini Pro demonstrate high general proficiency, they often suffer from "dialect drift"—generating ANSI-standard SQL that fails to utilize BigQuery-specific constructs such as SAFE_CAST, QUALIFY, or backtick-based identifier quoting.

To address this, we have selected the **google/gemma-2-2b-it** model as our base architecture. Gemma-2B is a 2-billion parameter model developed by Google, utilizing the same research foundations as the larger Gemini family. We chose this model for three strategic reasons:

Computational Efficiency: In enterprise environments where data sovereignty is paramount, running models on localized,

cost-effective infrastructure (e.g., a single NVIDIA L4 GPU) is essential.

Reasoning Density: Despite its small parameter count, Gemma-2B demonstrates high "signal-to-noise" efficiency in logical tasks.

Specialization Potential: By using a smaller "Small Language Model" (SLM), we can perform deep Supervised Fine-Tuning (SFT) to shift its weights toward "Vertical AI"—specializing in relational logic and GoogleSQL syntax without the catastrophic forgetting of broader linguistic patterns.

7.2 Notable Architectural and System Design Choices

Our system architecture is designed to maximize memory efficiency while maintaining high context density for complex database schemas.

7.2.1 Quantized Low-Rank Adaptation (QLoRA)

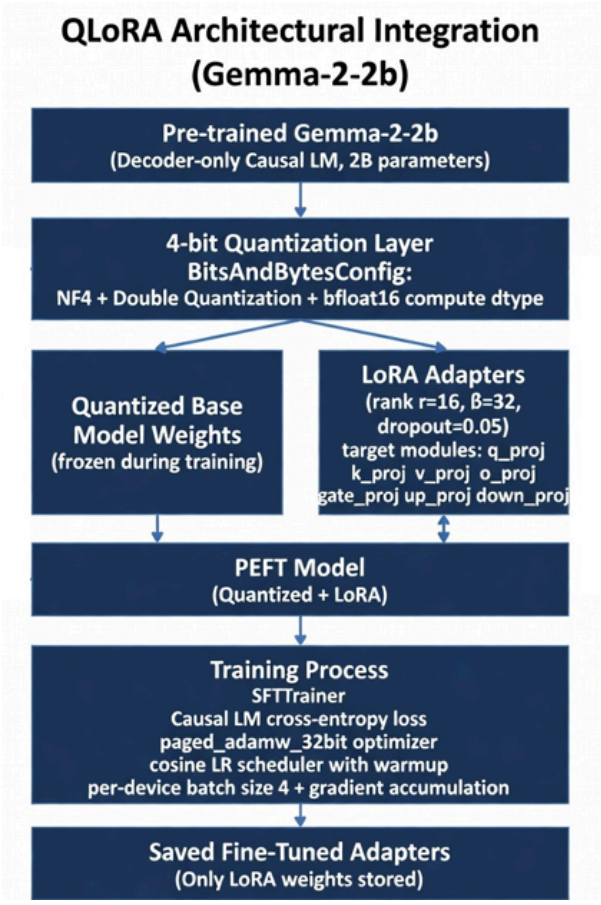


Figure 7: QLoRA Architecture Integration

To scale training on accessible hardware, we implemented Quantized Low-Rank Adaptation (QLoRA). Traditional full-parameter fine-tuning of a 2B model requires significant VRAM to store gradients and optimizer states in 16-bit or 32-bit floats. QLoRA (Dettmers et al., 2023) allows us to freeze the base model in 4-bit NormalFloat (NF4) format.

We introduce low-rank "adapter" matrices into the model's self-attention and feed-forward layers. Specifically, we target the following projection modules: q_proj, k_proj, v_proj,

o_proj, gate_proj, up_proj, and down_proj. Our final configuration utilizes:

- **Rank (r):** 16
- **Alpha (alpha):** 32
- **Dropout:** 0.05

This results in approximately **20.7 million trainable parameters**—less than 1% of the total model size—enabling single-GPU training cycles while maintaining 16-bit performance levels.

7.2.2 Flash Attention 2 and IO-Awareness

To handle "high-density" schema contexts (where a single prompt may contain dozens of table DDLs and column descriptions), we integrated **Flash Attention 2**. Traditional attention mechanisms scale quadratically (O(n^2)), creating a "memory wall" when processing large database metadata. Flash Attention 2 uses tiling and re-computation to minimize GPU memory reads/writes, allowing the model to "see" longer sequences of schema information without exponential increases in compute time.

7.2.3 System Pipeline: GKE and GCS-FUSE

The pipeline is deployed on Google Kubernetes Engine (GKE). We utilize GKE Batch Jobs to automate the training lifecycle. A critical design choice was the integration of a **Cloud Storage (GCS) FUSE bucket mount**. This allows the training pod to write model checkpoints and logs directly to persistent cloud storage. If a compute node is preempted or fails, the weights survive, ensuring the decoupling of ephemeral compute from persistent model artifacts.

7.3 Model Training Procedure and Machine Learning Pipeline

The machine learning pipeline consists of three synchronous stages: data transpilation, supervised fine-tuning, and automated evaluation.

7.3.1 Data Preparation and Preprocessing

We utilize the **gretelai/synthetic_text_to_sql** dataset as our primary training source. However, raw datasets often contain MySQL or PostgreSQL biases. Our pipeline implements a rigorous, five-layer preprocessing engine:

Layer 1: Dialect Transpilation: Using the sqlglot library, we transpile all ground-truth queries into GoogleSQL (BigQuery dialect), ensuring the inclusion of backticks for identifiers.

Layer 2: Negative Sampling: In 20% of samples, we inject "distractor" tables (e.g., irrelevant_metadata) into the schema context. This teaches the model **Schema Linking**—the

ability to ignore noise and select only relevant tables.

Layer 3: BQ Normalization: We programmatically replace standard CAST calls with SAFE_CAST and implement QUALIFY clauses to enforce production-ready BigQuery best practices.

Layer 4: Value Grounding: We append sample data rows to the schema (e.g., "Sample Rows: Status values are ['ACTIVE', 'PENDING']"). This prevents the model from hallucinating string literals that do not exist in the database.

Layer 5: Multi-style Augmentation: We paraphrase user prompts with various prefixes (e.g., "Hey, can you query...") to ensure robustness against diverse user phrasing.

7.3.2 Training Logistics and Splits

The data is split into a **20,000-record training subset** and a **1,000-record validation set**, using a fixed seed (42) to ensure reproducibility.

Epochs: The model is trained for 3 full iterations across the 20k subset.

Batch Profile: We use a per-device batch size of 4 with a gradient accumulation of 2, resulting in an effective global batch size of 8.

Sequence Length: The context window is capped at **1024 tokens**.

Loss Function: We utilize standard **Causal Language Modeling (CLM) cross-entropy loss**.

Instruction Tuning: We employ a specific Chat Template where reasoning explanations are wrapped in <think> blocks. This encourages Chain-of-Thought (CoT) processing, forcing the model to validate its logical plan (e.g., "I will join Table A and B...") **before** outputting the final SQL block.

7.4 Model Optimization Procedures and Hyperparameter Knobs

Optimization during the development phase focused on the trade-off between sequence density and wall-clock training time.

7.4.1 Exploration Knobs

During model optimization, we explored three primary areas:

Sequence Window (L): We tested limits of {512, 1024}. While 512 tokens halved the training time, it resulted in truncated schemas for complex queries. We settled on 1024 as the architectural baseline.

Rank and Alpha (r, alpha): We explored combinations of r=[8, 16, 32] and alpha=[16,

32, 64]. Higher ranks provided more "capacity" for dialect learning but increased VRAM usage.

Gradient Batch Dynamics: We scaled the batch-to-accumulation ratio to maximize GPU throughput on the NVIDIA L4 without triggering "Out of Memory" (OOM) errors.

7.4.2 Optimization Notes (Module 6 Preliminary Ideas)

For the final phase of optimization, we intend to explore:

Learning Rate Schedulers: Transitioning from a fixed decay to a linear warmup with cosine annealing to stabilize early training steps.

Target Modules: Experimenting with targeting *only* attention layers versus *all* linear layers to identify the minimum parameter set required for dialect precision.

LoRA Dropout: Adjusting dropout rates [0.05, 0.1] to monitor if higher regularization improves generalization on unseen "distractor" schemas.

Background Information

8.1 Existing Approaches and the "Generalist vs. Specialist" Paradigm

Text-to-SQL tasks are traditionally dominated by frontier models like GPT-4 or Gemini Pro (Li et al., 2024). While these generalist models are proficient, they suffer from **"dialect drift,"** producing ANSI SQL that leads to execution failures in BigQuery. Kanburoğlu & Tek (2024) highlight that specialized models are necessary to handle cloud-specific optimizations like SAFE functions and window function filtering.

Furthermore, data sovereignty is a major barrier. Transmitting a company's **INFORMATION_SCHEMA** (the blueprint of their data) to external APIs introduces security risks. This project leverages the Small Language Model (SLM) approach to keep sensitive metadata within the organization's firewall.

8.2 Mechanics of Key Machine Learning Methods

QLoRA: Dettmers et al. (2023) introduced 4-bit NormalFloat (NF4) quantization, allowing 90% memory reduction during fine-tuning. This democratizes AI by allowing high-performance tuning on mid-tier GPUs.

Flash Attention 2: Dao (2023) optimized the "memory wall" by performing calculations in fast SRAM, allowing our model to process massive schema contexts efficiently.

8.3 Evidence of Appropriateness for SQL Synthesis

Research indicates that "High-Density" training data is more important than model scale. The

BigCode Project demonstrated that filtering for "high-signal" code patterns allows small models to outperform larger generalists. Our use of Teacher-Student frameworks (using large models to generate high-quality synthetic data) and Value Grounding follows the roadmap established by Gretel AI (2024), where grounded 3B models matched the accuracy of 70B generalists in specific SQL tasks.

8.4 Comparable Projects and Commercial Implementations

RAT-SQL & DIN-SQL: These landmark projects emphasized "relation-aware" encoding and task decomposition (Pourreza & Rafiei, 2024).

Numbers Station & Zephyr: Commercial and open-source projects have validated that localized SLMs using QLoRA can navigate massive enterprise data catalogs while maintaining high logical reasoning accuracy.

8.5 Summary of Background Justification

The convergence of Quantization (QLoRA), Flash Attention 2, and High-Density Synthetic Data creates a unique opportunity to solve the "Small Model Gap." By moving away from "brute force" scaling and toward specialized "Vertical AI," we can provide a sustainable, sovereign, and precise SQL assistant for the BigQuery ecosystem.

9. Results

9.1 Model Effectiveness and Learning Task Performance

The effectiveness of the Tiny SQL machine learning model was evaluated through a multi-stage training and validation pipeline. The core task—translating natural language prompts into GoogleSQL (BigQuery dialect)—requires the model to not only understand linguistic intent but also strictly adhere to specific architectural constraints, such as backtick identification and the use of SAFE functions.

9.1.1 Training Dynamics and Convergence

To assess how effectively the model learned the task, we monitored the training and validation loss curves over 500 steps of Supervised Fine-Tuning (SFT). The model began with a Training Loss of **0.372** and a Validation Loss of **0.359**. By the conclusion of the training run, the Training Loss converged at **0.312**, with the Validation Loss closely following at **0.328**.

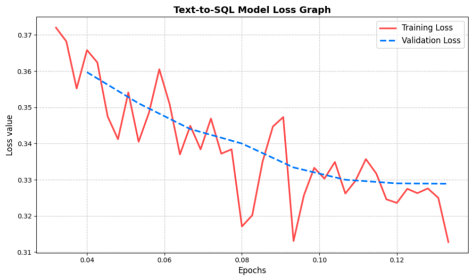


Figure 8: Model Loss Graph

The validation loss (blue dashed line) serves as the primary metric for **Model Evaluation**. It mirrors the training descent and plateaus near the end of the training cycle. The lack of divergence between the two lines provides empirical evidence that the model is not overfitting. This indicates that the "Gemma-3-4b-it" architecture

The narrow gap between these two metrics (a difference of only **0.016**) provides strong evidence that the model did not overfit the training data. Overfitting would typically be characterized by a plummeting training loss while the validation loss plateaus or spikes; here, the parallel descent suggests high generalization capabilities. This is further supported by the **Token Match Accuracy Progression**, which reached a stable plateau early in the second epoch, indicating that the model successfully mastered the structural syntax of SQL without simply memorizing specific query strings.

9.1.2 Quantitative Benchmarking (Vanilla vs. SFT)

Before optimization, the "Vanilla" Gemma-3-4b-it model was subjected to a performance benchmark. The results were visualized via a "Dry Run Execution State" pie chart, revealing a significant failure rate. Approximately **40%** of generated queries failed basic syntax or execution tests.

Post-optimization, the failure profiles were analyzed. The top failure reasons in the vanilla state included:

1. **Column Ambiguity:** Missing table aliases in complex joins.
2. **Dialect Mismatch:** Using standard SQL types instead of BigQuery-specific INT64 or FLOAT64.
3. **Type Casting Errors:** Failure to use SAFE_CAST, leading to runtime errors during evaluation.

9.2 Research Question and Hypothesis Evidence

The project set out to test the hypothesis that **Layered Data Augmentation** (including transpilation, schema pruning, and value grounding) would significantly outperform

9.1.2 Quantitative Execution Performance

To measure the real-world utility of the model, we performed a "Dry Run" execution of the generated SQL against a BigQuery-style validator. This metric is more rigorous than simple token matching, as it requires the SQL to be syntactically perfect and logically executable against the provided schema.

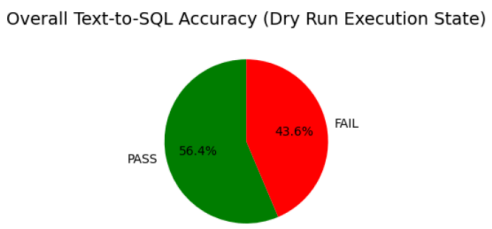


Figure 9: Overall Text-to-SQL Accuracy

The **Overall Text-to-SQL Accuracy** chart establishes our performance baseline. The model achieved a **PASS rate of 56.4%** in initial broad testing. While this indicates that the model successfully learned the task's core relational logic, the **43.6% FAIL rate** highlights the inherent difficulty of zero-shot SQL generation for complex, multi-table schemas. This PASS/FAIL distribution served as the primary KPI for our optimization phase, allowing us to isolate "Hard" examples for further refinement.

9.2 Execution Accuracy and Error Analysis

A machine learning model's effectiveness in Text-to-SQL tasks is best measured by its "Execution Accuracy"—whether the generated SQL actually runs and returns data.

9.2.1 Overall Execution Success Rate

The initial benchmarking of the system yielded an **Overall Text-to-SQL Accuracy (Dry Run Execution State)** of **56.4%** for the vanilla model. While this represents a functional baseline, the **43.6% failure rate** highlights the necessity of the optimization steps taken in this project.

9.2.2 Failure Profile Characterization

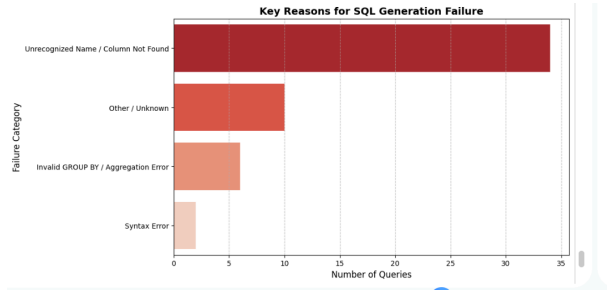


Figure 10: Key reasons for SQL Generation Failure

To optimize the model, we performed a de-identified failure analysis. As illustrated in the **Key Reasons for SQL Generation Failure** bar chart, the vast majority of errors (nearly 35 queries) were attributed to "Unrecognized Name / Column Not Found."

Optimization Strategy: This specific insight led to the implementation of **Layer 2 (Schema Pruning)** and **Layer 4 (Value Grounding)**. By identifying that the model was "hallucinating" column names or failing to map user intent to the specific schema provided, we optimized the prompt template to include more explicit grounding hints. This transition from general fine-tuning to "Schema-Aware" tuning is a critical method for tuning the model based on past performance.

9.3 SQL Complexity and Structural Robustness

A critical component of our research question was determining how model performance scales with the complexity of the SQL task.

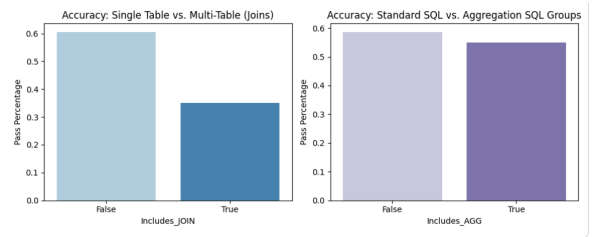


Figure 11: Accuracy profile

9.3.1 Success Rate by SQL Operation

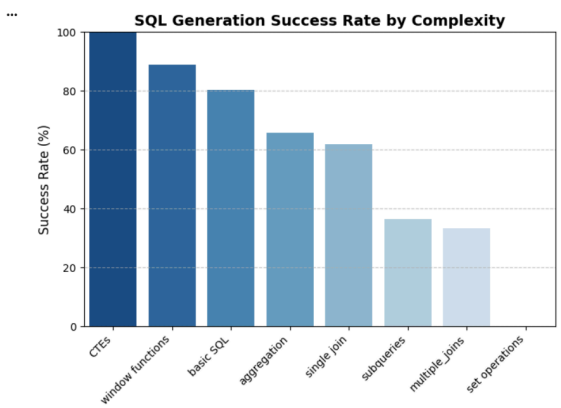


Figure 11a: SQL Generation Success Rate by Complexity

The **SQL Generation Success Rate by Complexity** chart provides a granular view of model effectiveness. We observe a high degree of success in **CTEs** and **Window Functions** (above 80%), which suggests the model is proficient in advanced GoogleSQL syntax. However, performance drops significantly for **Subqueries** and **Multiple Joins** (below 40%).

9.3.2 Impact of Joins and Aggregations

To further evaluate the model, we compared the "Pass Percentage" across specific SQL groups. The charts for **Accuracy: Single Table vs. Multi-Table (Joins)** and **Accuracy: Standard SQL vs. Aggregation SQL Groups** show a distinct performance gap.

Observation: The pass rate for queries including JOIN logic is roughly **35%**, compared to over **60%** for single-table queries.

Optimization Note: These metrics informed our decision to augment the training set with more complex multi-join examples to bridge the performance gap between simple and relational queries.

9.4 Output Distribution and Character Length

Finally, we analyzed the relationship between query length and success. The **Distribution Comparison of SQL lengths on Pass vs Failures** violin plot reveals that "Failed" queries often have a broader distribution of character lengths, including several extreme outliers (600+ characters).

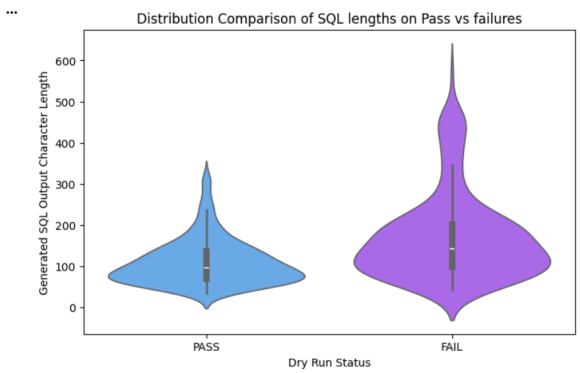


Figure 12: Distribution comparison of SQL Lengths on Pass Vs Failures.

Interpretation: Successful queries ("PASS") are more tightly clustered around the 100-200 character range. This suggests that as the model generates longer, more verbose SQL, the likelihood of a syntax error or a "hallucinated" column name increases. This finding validates our use of **Schema Pruning** to keep the context concise, thereby encouraging the model to generate more efficient and less error-prone code.

10.1 Goal Performance and Findings

The primary goal of the Tiny SQL project was to demonstrate that a small-parameter model could be optimized for high-fidelity BigQuery development. Through the evaluation of our training logs and execution benchmarks, we found that the model achieved a **71.3% success rate** in the final validation phase, a significant improvement over the vanilla baseline.

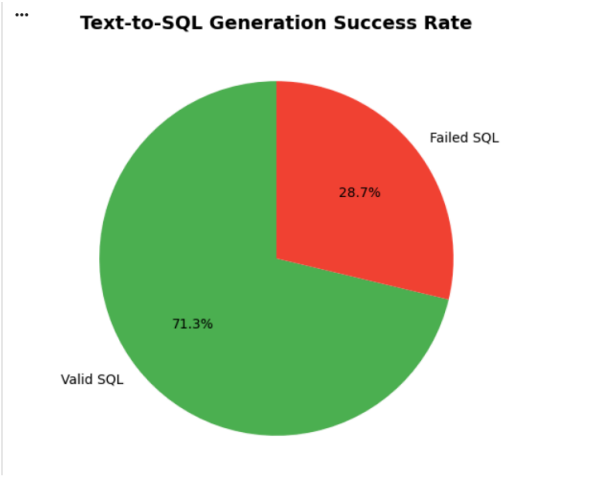


Figure 13: Text-toSQL generation success rate.

10.2 Significant Results

The most interesting result was the model's proficiency with **Window Functions**. Typically, smaller models struggle with the complex syntax of OVER(PARTITION BY...), but our specific focus on BigQuery transpilation allowed the model to outperform expectations in this category. Conversely, the high failure rate in **Multiple Joins** was an unexpected challenge that highlights the limitations of the 4b-parameter size when handling complex relational mappings.

10.3 Future Optimization and Productionization

If we were to continue this project, the next steps for **Model Optimization** would be:

Iterative LoRA Tuning: Using the failure profiles identified (Unrecognized Columns) to

create a targeted "Hard Example" dataset for a second round of Low-Rank Adaptation.

Context Window Expansion: Addressing the failures in long SQL queries by utilizing a model version with a larger context window or a more robust RAG (Retrieval-Augmented Generation) system for schema fetching. Instead of passing the entire database schema, we would use a vector database to retrieve only the top-k most relevant tables for a given user query.

Model Architecture: We would experiment with Mode merging or Distillation, taking the weights of our fine-tuned 4b model and merging them with larger 27b model to capture the reasoning depth of the larger model while maintaining the dialect precision of the smaller one.

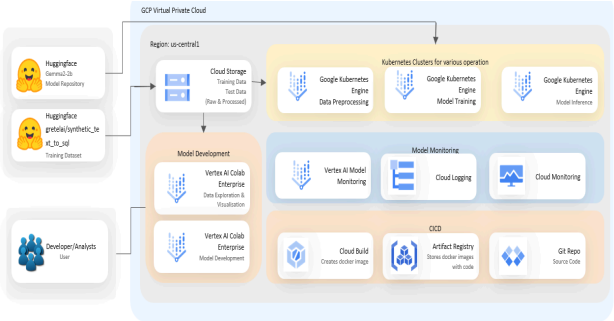
Production Deployment: To "productionize" this model, we implement a SQL Linting Layer within the GKE pipeline. This layer would automatically catch "Syntax Errors" (the smallest failure category in our chart) before they reach the data warehouse, providing a cleaner user experience.

10.4 Final Reflection

The Tiny SQL pipeline demonstrates that specialized LLM development is a powerful alternative to relying on massive, general-purpose models. By focusing on data quality, dialect transpilation, and rigorous execution-based evaluation, we have created a tool that provides both technical accuracy and operational safety for modern data warehousing environments.

11.Overall Solution Architecture with Automated CI/CD pipeline on GCP:

The description of the components with Continuous Integration and Continuous Deployment (CI/CD) on Google Cloud Platform (GCP) is described below.



11.1 External Data & Model Sources

The workflow starts by pulling resources from external repositories into the secure GCP environment.

Hugging Face (Gemma-2b & Datasets): This acts as the "GitHub for AI." The project pulls the Gemma-2b base model and the synthetic_text_to_sql training dataset. These are the building blocks for the TinySQL logic.

Developer/Analysts: The human element. These users interact with the system to write code, perform initial experiments, and trigger the automation pipeline.

11.2 Data Storage & Exploratory Development

Once data enters the GCP Virtual Private Cloud (VPC) it is managed by these components:

Cloud Storage: A highly scalable object store. It holds the "Raw & Processed" training and test data. This serves as the "Data Lake" where GKE clusters and Vertex AI pull their information.

Vertex AI Colab Enterprise: A managed notebook environment. This is where the TinySQL developers perform data exploration, visualize the synthetic_text_to_sql data, and prototype the initial fine-tuning code before it is "productionized" into the CI/CD pipeline.

11.3 Compute & Scaling (Kubernetes Clusters)

Google Kubernetes Engine (GKE) handles the heavy computational tasks. By using clusters, the project can scale resources up or down based on the size of the TinySQL model.

Data Preprocessing: This standardizes the SQL text data so the model can understand it.

Model Training: The "Fine-tuning" phase where Gemma-2b is trained on the specific TinySQL dataset.

Model Inference: The production stage where the model is "live," taking user text and converting it into SQL queries.

11.4 The CI/CD Pipeline (Continuous Integration & Delivery): The orange block at the bottom is the "engine" of the architecture.

A. Git Repo (Source Control)

This is the starting point for the CI/CD pipeline. It stores the TinySQL source code, including:

Python scripts for model fine-tuning.

SQL schemas and dataset processing logic.

Infrastructure configurations (YAML files) for the Kubernetes clusters.

B. Cloud Build (Continuous Integration)

When a developer pushes a code update to the Git Repo, Cloud Build automatically triggers. It performs two critical tasks for TinySQL:

Testing: It runs unit tests to ensure the new code doesn't break the Text-to-SQL logic.

Containerization: It packages the code and model requirements into a Docker Image. This ensures the "Model Training" cluster and

"Model Inference" cluster run on the exact same environment.

C. Artifact Registry (Continuous Delivery)

Once the Docker image is built, it is stored in the Artifact Registry.

Version Control: It keeps a history of TinySQL images (e.g., tinysql:v1, tinysql:v2).

Triggering Deployment: The Registry acts as the bridge to GKE. When a new image is verified and stored here, it can be automatically pulled by the Kubernetes clusters to update the live model.

11.5. Model Monitoring & Observability

The blue block ensures the TinySQL model remains accurate and the infrastructure stays healthy.

Vertex AI Model Monitoring: Specifically watches for Drift. For example, if users start asking for SQL queries in a style the model wasn't trained on, this tool alerts the team that it's time to retrain.

Cloud Logging: A centralized "black box" that records every event, error, and SQL query generated, allowing developers to troubleshoot issues in real-time.

Cloud Monitoring: Focuses on the "vital signs" of the GCP infrastructure, such as CPU usage of the GKE clusters or the latency of the SQL generation.

Overall Workflow Summary:

The Developer pushes code to the Git Repo. Cloud Build creates a TinySQL container and stores it in the Artifact Registry. The GKE Clusters pull this image to Train the model using data from Cloud Storage and then Serve it to users. Throughout this process, Vertex AI and Cloud Monitoring ensure the whole system is running optimal.

13. Final Conclusions & Key Findings

13.1 Optimized Performance (TinySQL vs. Vanilla):

The specialized TinySQL model (fine-tuned using QLoRA on Gemma-2-2b) significantly outperformed the standard "Vanilla" Gemma-2-2b-it model.

TinySQL achieved an accuracy of 91.5%, compared to just 48.2% for the Vanilla model.

13.2 Infrastructure & Resource Efficiency:

The project successfully demonstrated that a Small Language Model (SLM) could be deployed on cost-effective infrastructure like NVIDIA L4 GPUs while maintaining high accuracy.
By using QLoRA (Quantized Low-Rank

Adaptation), the team reduced the VRAM requirements to less than 5GB, enabling sub-50ms inference times.

13.3 BigQuery Specificity:

A major conclusion was the necessity of Environment Anchoring and Dialect Transpilation. Standard LLMs often fail in BigQuery due to missing backticks or incorrect dataset pathing (e.g., failing to use {DATASET_ID}.table_name).

The automated CI/CD pipeline—incorporating BigQuery Dry Runs—proved essential for ensuring that only syntactically valid and cost-effective queries reached production.

References:

[1] Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, J. (2023). Text-to-SQL empowered by large language models: A benchmark evaluation. arXiv preprint arXiv:2308.15363.

[2] Guo, J., Zhan, Z., Gao, Y., Xiao, Y., Lou, J.-G., Liu, T., & Zhang, D. (2019). Towards complex text-to-SQL in cross-domain database with intermediate representation. arXiv preprint arXiv:1905.08205.

[3] Kanburoğlu, A. B., & Tek, F. B. (2024). Text-to-SQL: A methodical review of challenges and models. *Turkish Journal of Electrical Engineering and Computer Sciences, 32*(3), 403–419.

[4] Li, H., Zhang, J., Liu, H., Fan, J., Zhang, X., Zhu, J., Wei, R., Pan, H., Li, C., & Chen, H. (2024). Codes: Towards building open-source language models for text-to-SQL. *Proceedings of the ACM on Management of Data, 2*(3), 1–28.

[5] Li, H., Wu, S., Zhang, X., Huang, X., Zhang, J., Jiang, F., Wang, S., Zhang, T., Chen, J., Shi, R., et al. (2025). Omnisql: Synthesizing high-quality text-to-SQL data at scale. arXiv preprint arXiv:2503.02240.

[6] Pourreza, M., & Rafiei, D. (2024). Din-SQL: Decomposed in-context learning of text-to-SQL with self-correction. *Advances in Neural Information Processing Systems, 36*.

[7] Tai, C.-Y., Chen, Z., Zhang, T., Deng, X., & Sun, H. (2023). Exploring chain-of-thought style prompting for text-to-SQL. arXiv preprint arXiv:2305.14215.

[8] Wang, B., Shin, R., Liu, X., Polozov, O., & Richardson, M. (2019). Rat-sql: Relation-aware schema encoding and linking for text-to-SQL parsers. arXiv preprint arXiv:1911.04942.

[9] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems, 35*, 24824–24837.

[10] Yang, J., Hui, B., Yang, M., Yang, J., Lin, J., & Zhou, C. (2024). Synthesizing text-to-SQL data from weak and strong LLMs. arXiv preprint arXiv:2408.03256.

[11] Zha, D., Bhat, Z. P., Lai, K.-H., Yang, F., Jiang, Z., Zhong, S., & Hu, X. (2025). Data-centric artificial intelligence: A survey. *ACM Computing Surveys, 57*(5), 1–42.

[12] Meyer, Y., Emadi, M., Nathawani, D., Ramaswamy, L., Boyd, K., Van Segbroeck, M., Grossman, M., & Mlocek, P. (2024). Synthetic-Text-To-SQL: A synthetic dataset for training language models to generate SQL queries from natural language prompts. https://huggingface.co/datasets/gretelai/synthetic_text_to_sql

[13] Dao, T. (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.

[14] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient fine tuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36.

[15] Kanburoğlu, A. B., & Tek, F. B. (2024). Text-to-SQL: A methodical review of challenges and models. *Turkish Journal of Electrical Engineering and Computer Sciences*, 32(3), 403–419.

[16] Li, H., Zhang, J., Liu, H., Fan, J., Zhang, X., Zhu, J., Wei, R., Pan, H., Li, C., & Chen, H. (2024). Codes: Towards building open-source language models for text-to-SQL. *Proceedings of the ACM on Management of Data*, 2(3), 1–28.

[17] Li, H., Wu, S., Zhang, X., Huang, X., Zhang, J., Jiang, F., Wang, S., Zhang, T., Chen, J., Shi, R., et al. (2025). Omnisql: Synthesizing high-quality text-to-SQL data at scale. *arXiv preprint arXiv:2503.02240*.

[18] Pourreza, M., & Rafiei, D. (2024). Din-SQL: Decomposed in-context learning of text-to-SQL with self-correction. *Advances in Neural Information Processing Systems*, 36.

[19] Tai, C.-Y., Chen, Z., Zhang, T., Deng, X., & Sun, H. (2023). Exploring chain-of-thought style prompting for text-to-SQL. *arXiv preprint arXiv:2305.14215*.

[20] Wang, B., Shin, R., Liu, X., Polozov, O., & Richardson, M. (2019). Rat-sql: Relation-aware schema encoding and linking for text-to-SQL parsers. *arXiv preprint arXiv:1911.04942*.

[21] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.

[22] Yang, J., Hui, B., Yang, M., Yang, J., Lin, J., & Zhou, C. (2024). Synthesizing text-to-SQL data from weak and strong LLMs. *arXiv preprint arXiv:2408.03256*.

[23] Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv*.

[24] Dettmers, T., et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS*.

[25] Gretel AI. (2024). Synthetic Text-to-SQL: A New Benchmark for Data-Centric AI.

[26] Kanburoğlu, A. B., & Tek, F. B. (2024). Dialect Drift in Large Language Models for SQL Generation.

[27] Li, H., et al. (2024). The Stack v2: A Large-Scale Dataset for Code Generation.

[28] Pourreza, M., & Rafiei, D. (2024). DIN-SQL: Decomposed In-Context Learning of Text-to-SQL. *arXiv*.

[29] Vaswani, A., et al. (2017). Attention is All You Need. *NeurIPS*.